Technologisches Gewerbemuseum

Höhere Technische Lehranstalt für Informationstechnologie
Schwerpunkt Systemtechnik

Diploma Thesis

Helikon

State of the Art, Machbarkeit & Konzept

Oliwier Przewlocki 5BHIT

Betreuer:in: Doktor Dominik Dolezal MSc
Performed in the school year 2025/26

Submission note:
December 22, 2025 Approved:

Statutory declaration

I declare that I have authored this thesis independently, that I have not used other than the declared sources and that I have explicitly marked all material which has been quoted either literally or by content from the used sources. I also used the following generative AI tools [e.g. ChatGPT, Grammarly Go, Midjourney] for the following purpose:

- Perplexity: Research of scientific papers
- ChatGPT: Sentence formulation, synonyms, brainstorming, diagram generation

Wien, 22.12.25

Location, Date

Przewlocki

Oliwier Przewlocki

Contents

Chapter 1

State of the Art

1.1 Embedding Technologies in Modern Retrieval Systems

Choosing an embedding model has a direct impact on the quality, latency and robustness of retrieval. Since literature research already covered the theoretical foundations of embeddings and semantic searches, this section will focus on the current technologies that are being used in practice.

1.1.1 Overview of current embedding model families

Embedding models can be generalized into several major families, with each having distinct trade-offs between performance, deployment, etc.

- **SBERT variants** are an early adoption of transformer-based embedding models that are still widely used. It modifies the BERT architecture to allow efficient vector generation that span large documents [1]. SBERT's strength lies in the balance between accuracy and cost, and it can be deployed without GPU acceleration [1]. Although the performance of SBERT was surpassed by more recent models, it remains a stable and predictable option for systems where offline support and/or resource constraints are necessary [1].
- **E5 models** are specifically designed to consistently perform well on retrieval-based tasks and rank among the top performers on the MTEB (Massive Text Embedding Benchmark) [2]. Unlike SBERT, the E5 architecture specifically use instruction-tuned embeddings by default, which enables vast generalization across many query types without manual engineering [3]. The models can run completely on-premise and require no proprietary infrastructure as support, which is ideal for privacy-focused environments such as the Helikon system [3]. Their balance between speed, performance and open-source availability makes E5 one of the most suitable choices for modern RAG applications [2].
- **Instructor models** introduce *instruction-conditioned embedding generation*, which adds instruction prefixes that the text is encoded together with. Examples of such prefixes would be "query:", "document:", etc. This enables the model to generate embeddings that are fine-tuned for specific tasks [4]. Instructor models excel at retrieving different text types that require divergent semantic handling [4]. However, the instructions introduce heightened operational complexity, since each embedding must be formatted with the correct instruction template [4]. This causes the models to be more powerful, but less straightforward to integrate into a pipeline.
- **ADA-002** are cloud-based models that offer high-quality embeddings with a strong benchmark performance [5]. They are easy to integrate and require no local infrastructure. Furthermore, they receive constant provider-side updates [5]. However, third-party cloud-based infrastructure comes with several major drawbacks. Mandatory cloud connection could raise privacy concerns, the usage cost may increase over time and reliance on a single provider is never recommended since it creates a vendor lock-in [6].

1.1.2 Technical comparison dimensions:

Choosing the optimal embedding model doesn't only involve accuracy. Several technical factors define the cost, long-term cost, and viability of the system. This is especially true for educational settings where previously named resource constraints and privacy concerns are ever present.

A major dimension to consider are hardware requirements. Lightweight models, such as SBERT, are designed with efficient CPU execution in mind, without the need for GPU acceleration, which enables consumer-grade machines to use the models effectively [1]. In contrast, models such as E5-large or

Instructor-XL demand considerably more memory and processing power to achieve optimal retrieval performance [3]. This in turn makes them less suitable for minimal setups or offline usage. Cloud-based embeddings such as ADA-002 fully delegate the computation task to OpenAI's infrastructure. This removes the need for powerful local hardware during retrieval, but introduces a major dependency on OpenAI's third-party services and a stable internet connection [5].

Another important factor is vector dimensionality and its impact on memory usage. Models may produce embeddings ranging from 384 to over 1024 dimensions, depending on the architecture [3]. While higher-dimensional vectors often contain richer semantic features, they drastically increase the size of in-memory indexes, which slows down the similarity search and raises the hardware requirements for vector databases like ChromaDB or FAISS, which are analyzed in detail in the upcoming section on vector storage (Section 1.2) [6]. In practice, the dimensionality must balance between power and scalability.

Latency-accurate trade-offs also play a major role in system design. Instructor models generally take the lead on retrieval benchmarks, but their reliance on previously mentioned prefixes and larger architectures results in an increased pipeline engineering complexity and higher inference times [4]. Faster models like SBERT or *nomic-embed-text* offer lower latency but fall behind in complex query understanding and domain-specific recall.

Finally, licensing and cost have a large effect on the deployment process and its flexibility. Open-source models, such as SBERT and E5, can be fully self-hosted and require no further architectural dependencies. This offers complete control over the data flow and compliance with GDPR [7]. In contrast, proprietary services, e.g. ADA-002, require querying from third-party APIs, bringing along usage fees and introducing vendor lock-in risks [6].

The technical trade-offs described above build directly on the theoretical principles of vectorization and embedding space representations that were introduced in the literature research. Concepts such as semantic similarity, dimensionality and cosine-based retrieval remain in focus, but they are analyzed from a system integration perspective.

1.2 Vector Databases and Similarity Search Technologies

Vector databases form the backbone of embedding-based retrieval systems by storing and indexing high-dimensional representations of textual or multimedia data. Unlike traditional relational databases, they are optimized for approximate nearest neighbor (ANN) search in continuous vector spaces. Their internal design influences retrieval speed, memory usage, scalability, and filtering precision. As such, choosing an appropriate vector store is critical for building responsive and domain-adaptable pipelines. The following section provides a technical overview of prominent vector database frameworks and analyzes their core indexing and filtering capabilities [8].

1.2.1 Description of leading vector DBs

Vector databases play a critical role in modern retrieval pipelines, serving as the storage layer for embeddings. Their ability to search, index and filter directly influences speed, scalability and privacy of the retrieval process. Depending on the context, different vector stores offer different trade-offs in terms of performance, complexity, etc.

- **ChromaDB** is an offline-focused, lightweight vector database for fast prototyping and small-scale applications. It uses DuckDB under the hood for persistence and provides a Python API that allows for seamless integration with frameworks such as LangChain (Section 1.3.1) [9]. The database stores indexes in-process and doesn't require complex infrastructure, which makes it useful in educational projects where simplicity and offline availability are a priority [9]. While it offers HNSW indexing by default, it lacks support for alternative ANN indexing strategies [9]. Its architecture is ideal for systems where moderate performance suffices and local data control is essential.
- **Weaviate** is a distributed, open-source vector search engine designed for scalable, multi-tenant applications. Unlike lightweight solutions like ChromaDB, it offers a wide range of features including hybrid search (combination of dense and sparse retrieval), GraphQL-based filtering and scalable modules for vectorization and storage [10]. It supports both local and managed deployments, making it a robust solution across a wide range of scenarios. Its support for metadata filtering makes it particularly useful for educational tools or platforms handling segmented user bases [10]. Compared to ChromaDB, Weaviate requires more infrastructure but provides more flexibility and performance tuning.
- **Pinecone** is a managed vector database offered as a cloud SaaS product. It drastically reduces infrastructure complexity and provides low-latency approximate nearest neighbour (ANN) search by default [11]. Its APIs support advanced filtering and provide namespace support, but all operations are bound to Pinecone's remote cloud platform. This in turn creates potential vendor lock-in with the addition of issues including pricing scalability and data residency [11]. While it's a strong option for commercial applications that benefit from delegating storage and requiring minimal maintenance, its lack of local hosting option makes it less optimal for academic, offline, or privacy-focused environments.
- **FAISS** (Facebook AI Similarity Search) is a low-level indexing library focusing on similarity search, often used as a base component in other vector databases. It provides efficient key indexing strategies, such as IVF and HNSW. Additionally, FAISS is optimized for both CPU and GPU environments [12], making it robust and scalable. Unlike Weaviate or ChromaDB, FAISS isn't a full-featured database, as it lacks metadata filtering, storage persistence and access control mechanisms. However, its performance make it a well respected choice in custom setups where retrieval logic is coupled to the application layer. Many modern vector engines (e.g. Qdrant, Milvus) use FAISS for ANN search acceleration [12].

1.2.2 Technical features compared

The following comparison underlines the technical dimensions that are relevant for vector database system evaluation in practice.

Indexing algorithms play a central role in performance but aren't always exposed to the user in the same way. FAISS supports multiple indexing strategies, including Flat, IVF, and HNSW and is known for its low-level configuration options [12]. Pinecone and Weaviate primarily abstract away these choices, with a small exception for advanced Weaviate users who get additional HNSW configurations [10]. ChromaDB defaults to HNSW without exposing alternatives, which favors simplicity over performance tuning [9].

Metadata filtering and namespace partitioning are strongly linked to optimized scoped retrieval. Pinecone and Weaviate have built-in metadata filters that can be combined with search queries, allowing for fine-tuned control over the result sets based on fields such as tags or categories [10] [11]. Pinecone also provides namespaces for separation at an index level, whereas ChromaDB only provides basic metadata support and lacks namespace isolation. FAISS, as said before, isn't designed to be a fully functional database and doesn't natively support metadata queries [12].

Multi-tenancy and tenant isolation are essential in educational and SaaS scenarios where users or classes shouldn't share retrieval contexts. Weaviate provides tenant-level separation, which keeps data isolated across "shards", enabling secure multi-user indexing and ensuring a stable and secure shared infrastructure [10]. Pinecone can simulate multi-tenancy with namespaces, but queries cannot span across them and custom permissioning solutions need to be implemented externally [11]. Both ChromaDB and FAISS don't support tenant isolation, although there exist workarounds using collection naming.

Latency, throughput & scalability range significantly across different systems. FAISS ensures high throughput with support for GPU acceleration, making it optimal for large datasets on optimized hardware [12]. Although Pinecone abstracts performance tuning, it also provides results with low latency at a large scale [11]. Weaviate supports both horizontal scaling and distributed storage, with the ability to shard data and replicate across different nodes, ensuring high availability [10]. In contrast, ChromaDB is designed for single-node, low-traffic environments and is best suited for lightweight and prototyping setups [9].

Deployment models determine how flexible the system integration is. FAISS and ChromaDB are both open-source and can be used in any local or server-based workflow. Weaviate supports both cloud-managed and on-premise deployment using Docker. Pinecone is cloud only, with no current available option to self-host, making it the least flexible among the options compared [11]. The choice between local, hybrid, or SaaS deployment largely stems from project requirements around data control, compliance and infrastructural scaling.

1.3 Frameworks for Building Retrieval-Augmented Generation Pipelines

Retrieval-augmented generation (RAG) pipelines enable large language models to access external knowledge without retraining by integrating retrievers and a generator into a unified loop. A user query is embedded, semantically matched in a vector store, and used to ground the model's output. This architecture supports more controllable and interpretable systems and is particularly effective for applications where factual precision and modularity are required. In the following section, core infrastructure technologies behind the retrievers component—specifically vector databases—are analyzed in technical detail [13].

1.3.1 Overview of current frameworks

Several frameworks have emerged throughout the years to simplify the development of retrieval-augmented generation systems by abstracting complex logic and providing reusable components. These frameworks differ in architecture, flexibility, integration depth, and pipeline suitability. The following section provides an overview of the most commonly used open-source libraries for building RAG systems.

- **LangChain** is one of the most widely used frameworks for RAG applications. Its architecture utilizes modular "chains" to define data flow across retrievers, LLMs, buffers, output parsers and other components [14]. A major upside of LangChain is its ecosystem integration. It supports vector databases like ChromaDB, FAISS, Weaviate and Pinecone, as well as embedding models from OpenAI, HuggingFace, etc [14]. Furthermore, it includes tools for agent routing, document loaders, and conversation memory, making it a great option for chatbot and assistant-like applications [14]. Additionally, its open-source approach, extensive docs and frequent updates make it one of the most mature options for educational and production-ready RAG implementations [14].
- **LlamaIndex**, previously known as GPT Index, provides an alternative focused on flexible index definitions and composition. Instead of retrieval generation through tool pipelines, LlamaIndex uses high-level abstractions like vector indexes, keyword tables and graph indexes [15]. This allows developers to experiment with different strategies in context of retrieval, chunking, and hybrid architectures. It's well suited for experimenting where fast prototyping and customization are more important than a strict structure. Its design places an emphasis on readability and minimal configuration [15], but lacks some built-in integrations and orchestration abilities of other frameworks, such as LangChain [15].
- **Haystack** is a Python framework, developed by *deepset* that originally focused on extractive question answering but evolved into a full general-purpose RAG framework with strong support for transformer retrievers and LLMs. It emphasizes research-friendly pipeline architecture, enabling developers to assemble a wide range of components, such as document stores, retrievers and generators using a node-based configuration [16]. Haystack supports backends like Elasticsearch, FAISS and OpenSearch [16]. Compared to LangChain and LlamaIndex, Haystack leans towards academic workflows, offering more transparency and model control at the cost of a steeper learning curve [16].

1.3.2 Technical comparison dimensions

The choice of a framework depends not only on its architectural model but on its integrations with external components, its extensibility and real-world applicability.

Integration with embedding models and vector databases varies across the different frameworks. LangChain offers the broadest support, with native wrappers for embedding models from e.g. OpenAI, Cohere and HuggingFace. It seamlessly integrates with vector stores like ChromaDB, Pinecone and Weaviate [14]. LlamaIndex provides a similar level of support, but introduces a proprietary level of abstraction over indexes, allowing for hybrid combinations of keyword, vector and graph-based retrieval [15]. In contrast, Haystack focuses more on FAISS, Elasticsearch and OpenSearch. Additionally, it supports HuggingFace and custom transformers for embeddings [16].

Complexity vs. ease of use is a core trade-off. LangChain's flexibility costs configuration overhead, as component chaining often involves detailed setups and understanding of internal interfaces [14]. LlamaIndex has a much simpler approach, allowing developers to load and query documents with minimal setup, which in turn creates a less customizable environment at the cost of reduced pipeline granularity [15]. Haystack, which was originally targeted towards researchers, favors explicit and detailed configuration over over abstraction. This results in a steeper learning curve but better control over model behavior and input-output flow [16]. For beginners and projects requiring quick prototypes, LlamaIndex tends to offer an easier entry point, while LangChain scales better into fully-fledged applications [5].

Stability, community size & documentation largely influence maintainability. LangChain has the largest open-source community amongst the previously mentioned frameworks, with active development, a wide range of plugins and frequent updates [14]. Its documentation is extensive but also sometimes inconsistent due to frequent API updates. Similar to LangChain, LlamaIndex also maintains a fast release cycle, though its API is more stable and easier to follow [15]. Haystack, while less active than LangChain, is well-documented and particularly strong for research use cases, with tutorials, academic references and reproducible benchmarks [16].

In summary, LangChain, LlamaIndex and Haystack each offer different advantages depending on the use case. LangChain excels at modularity and integration, LlamaIndex focuses on simplicity and flexible index structures and Haystack provides a research-oriented, transparent pipeline for retrieval tasks [5]. The final choice for the framework depends on specific system requirements of in terms of ecosystem compatibility, developer workflow preferences, hardware and customization depth.

1.4 Security and Access-Control Technologies in Retrieval Systems

In multi-user environments, where RAG systems interact with sensitive content, enforcing strict access control mechanisms is a requirement [5]. These systems often store embeddings or documents that should only be visible to specific users or roles. Without specified permissioning and isolation, retrieval could leak protected information or violate data compliance such as GDPR [7].

1.4.1 Industry solutions

This section outlines current technical solutions for embedding access logic directly into the retrieval process, focusing on enterprise-grade implementations that solve these challenges in production.

- **QueryPie RAG 2.0** introduces one of the first publicly available blueprints for embedding accessibility policies in the retrieval pipeline itself [17]. In contrast to filtering based on policies after retrieval, QueryPie enforces its policies before any similarity search occur, which integrates permission checks directly into the ANN traversal. Consequently, this model ensures zero leakage risk and reduces overhead of post-processing sensitive data [17]. Additionally, it supports policy evaluations on a multi-attribute level (e.g. role, tag, etc.) making it suitable for dynamic permissioning. However, the system assumes the data is correctly annotated and may require vector pre-processing to align embeddings with access policies.
- **Weaviate's native multi-tenancy architecture** provides tenant-level, data isolation via index sharding [10]. Each tenant lives within a logically and physically separated vector space, supported by separate shards that prevent cross-access even at low levels. The vector index and metadata each tenant is associated with are not accessible by any other, and queries are automatically scoped to the tenant's context based on API tokens [10]. This enables real horizontal scaling while preserving retrieval integrity across schools, companies, or segmented user groups. In contrast to systems where filtering is enforced via external modules on the application level, Weaviate integrates this behavior natively, simplifying the configuration. One drawback is that it assumes that tenants are known and fixed at index creation, meaning that dynamic policies still need to be implemented externally.
- **Azure OpenAI** utilizes tenant-isolated architecture on the infrastrucural level [18]. When request are sent to OpenAI APIs through Azure, they're given dedicated deployment slots, network isolation and tenant-specific billing and monitoring. Although this doesn't solve semantic retrieval isolation

directly, it ensures LLM calls and data routing remain within an organization's scope [18]. E.g. prompts and completions never leave the customer's Azure boundary and remain hidden across tenants. This design is optimal for regulated environments, where data locality is enforced at the platform layer. However, it doesn't replace vector-level access control, so it needs an additional retrieval-layer isolation module, such as the ones discussed above for full RAG privacy.

1.4.2 Relevant Control Mechanisms

Modern retrieval systems need more than just basic security. They must enforce document-level permissions at runtime, comply with regulations and ensure complete isolation across sessions and tenants. This section covers the foundational control mechanisms used to enforce secure retrieval [6].

Metadata-based filtering before retrieval allows for a restricted scope using tags, fields and other attributes. Before a semantic search is executed, filters can be applied to exclude documents based on specific tags. Vector databases, such as Weaviate and Pinecone support hybrid queries where both metadata filters and similarity scores are combined to deliver relevant and permitted documents into the scoring pipeline [10] [11]. This improves both precision and data privacy by excluding vectors at query time.

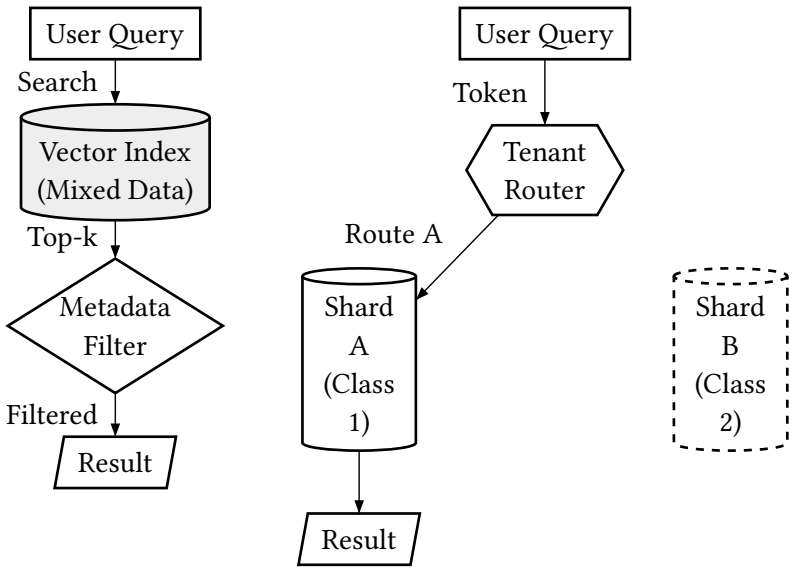

Figure 1.1: Comparison of Isolation Strategies: Shared Index with Metadata Filtering (Left) vs. Native Tenant Sharding (Right) [19]

As visualized in Figure 1.1, there is a fundamental architectural difference between these approaches. Standard metadata filtering (left) relies on query-time exclusion within a shared index, where all tenants' data resides together. In contrast, native tenant sharding (right) routes queries to physically separated data shards based on a tenant identifier. This physical separation provides a stronger security guarantee than logical filtering, as it eliminates the risk of cross-tenant data leakage during the vector search phase itself.

Policy enforcement introduces rules that are tied to user roles or session-based attributes. E.g. documents may only be accessed by users with the "*teacher*" role. This can either be modeled with role-based access control (RBAC) or attribute-based access control (ABAC), that were described

extensively in the literature research [20]. Implementing such policies in the retrieval pipeline trims the results off unauthorized vectors, which ensures that users only see what they're allowed to see. While systems like QueryPie RAG 2.0 implement this directly during the vector retrieval stage [17], other (e.g. LangChain) rely on upstream policy logic [14].

GDPR-compliant storage is also an important factor, especially in European contexts [7]. Personal data must be stored with encryption and proper access controls. Vector databases typically offer encryption at rest and in transit, and some provide regional hosting guarantees to comply with residency rules [6]. Furthermore, embeddings shouldn't contain any data that can be linked with identifiable personal content unless explicitly consented. Systems must support erasure requests by deleting or regenerating vector records tied to the user [7].

In both RBAC and ABAC, document visibility is determined not just by the data's content but by who the user is and what their query content is [20]. Retrieval systems that successfully integrate these policies at index or pipeline level ensure security and compliance in multi-user environments [6].

1.5 Summary of Technological Landscape

Secure retrieval systems must balance scalability, flexibility and privacy. The described architectures and mechanisms show that industry-ready access control is possible through metadata filtering, isolation policies, and GDPR-conform storage practices [6] [7].

Still, not all solutions offer equal trade offs. While QueryPie and Weaviate focus on native control integration [17] [10], other like Azure or LangChain-based stacks require additional logic and manual configuration [18] [14]. The next section will evaluate these alternatives not only on their security capabilities, but also on performance, deployment effort and suitability.

Chapter 2

Feasibility Study

2.1 Evaluation Methodology

The feasability study evaluates core technologies that are relevant to the development of Helikon's educational RAG prototype. Based on the previously defined requirements, technologies will be rated across a scoring model and will be summarized in table form. The goal is to find out an optimal technology selection that balances retrieval quality, performance, deployability and compliance.

The following criteria were chosen:

1. Retrieval quality (performance in retrieval tasks, using MTEB and related literature as a benchmark) [2]
2. Latency and scalability [6]
3. Integration complexity (ease of use with LangChain, ChromaDB and other pipeline tools) [14] [9]
4. GDPR compliance and data privacy (e.g. support for local processing or avoidance of vendor lock-in) [7]
5. Deployment constraints (local vs. cloud) [6]
6. Multi-tenancy and isolation [10]
7. Cost and licensing

Scoring is based on a 1–5 scale per criterion:

- 1 = poor/no support
- 3 = acceptable
- 5 = excellent/optimal

2.2 Feasibility of Embedding Models

The following embedding models were evaluated:

- SBERT (e.g. `all-mpnet-base-v2`, `all-MiniLM-L6-v2`) [1]
- E5 (e.g. `intfloat/e5-large-v2`) [2] [3]
- ADA-002 (`text-embedding-ada-002` by OpenAI) [5]

Model	Retrieval	Latency / Scale	Integration	Privacy	Deployment	Multi-Tenant	Cost	Total
SBERT	2	3	3	3	3	0	3	17
E5-base	3	2	3	3	3	1	3	18
ADA-002	3	3	3	1	0	0	1	11

Table 2.1: Evaluation of Embedding Models Based on Retrieval, Integration, and Deployment Factors

- **Retrieval Quality**: E5 leads with the highest MTEB scores (64.4%), outperforming SBERT and ADA-002 on average [2].
- **Latency & Scalability**: ADA-002 benefits from OpenAI's infrastructure, while E5 and SBERT require more local compute, especially at scale [5].
- **Integration**: All models integrate with LangChain and ChromaDB. ADA-002 offers easiest setup via API; SBERT and E5 require HuggingFace setup [14]
- **GDPR & Privacy**: SBERT and E5 are fully self-hostable, ideal for GDPR compliance. ADA-002 sends data externally, scoring lowest here [7].

- **Deployment**: ADA-002 wins in deployment ease (cloud API); SBERT is CPU-friendly; E5 needs GPU for acceptable speed [1] [3].
- **Cost**: SBERT and E5 are free to use (open-source), while ADA-002 incurs pay-per-token costs [5].
- **Suitability for Education**: E5′s multilingual and instruction-tuned structure make it ideal for classroom diversity [3].

Based on the weights in Table 2.1, E5-large (instruction-tuned) is selected as the most suitable embedding model for Helikon's RAG pipeline. It balances high retrieval performance, broad language support and full control over deployment [2] [3].

2.3 Feasibility of Vector Databases

The following vector databases were evaluated:

- ChromaDB [9]
- Weaviate [10]
- FAISS [12]
- Pinecone [11]

Database	Deployment	Latency / Scale	Multi-Tenant	Metadata Filtering	Integration	GDPR	Total
ChromaDB	3	2	1	2	3	3	14
Weaviate	3	3	3	3	3	3	18
FAISS	3	3	0	0	2	3	11
Pinecone	0	3	2	3	3	1	12

Table 2.2: Comparison of Vector Databases Based on Deployment, Isolation, and Framework Integration

- **Deployment**: ChromaDB and FAISS are fully self-hostable, ideal for local-first applications [9] [12]. Weaviate supports both on-prem and cloud deployments [10]. Pinecone is cloud-only and cannot be deployed offline [11].
- **Latency & Throughput**: Weaviate and Pinecone offer low-latency ANN search for production loads [10] [11]. ChromaDB is efficient for mid-size retrieval but lacks multi-threaded ANN acceleration. FAISS is highly performant and can be GPU-accelerated but requires manual configuration [12] [6].
- **Multi-Tenancy**: Weaviate provides native tenant isolation at the index level [10]. Pinecone supports namespace separation but lacks cross-namespace querying [11]. ChromaDB supports collection-level separation but no strict multi-tenancy model. FAISS doesn't support multi-tenancy [12].
- **Metadata Filtering**: ChromaDB, Weaviate, and Pinecone support document-level metadata filtering [9] [10] [11]. FAISS lacks native support for filter queries and requires wrapper implementations [6].
- **Integration**: All systems support LangChain integration. ChromaDB and Weaviate have mature LangChain modules. FAISS is mostly used as an internal index backend [14].

- **GDPR & Privacy**: ChromaDB and FAISS can run entirely locally, supporting full data sovereignty [7]. Weaviate can be self-hosted as well [10]. Pinecone is remote-only and tied to U.S.-based infrastructure [11].

Based on the score breakdown in Table 2.2, ChromaDB is selected as the best-fitting vector DB for the Helikon system [9]. It strikes the best balance between integration depth, local deployment, and educational simplicity making it ideal for a privacy-focused, low-infra RAG setup.

2.4 Feasibility of RAG Frameworks

The following RAG orchestration frameworks were evaluated:

- LangChain [14]
- LlamaIndex [15]
- Haystack [16]

Framework	Integration	Abstraction	Extensibility	Docs & Ecosystem	Metadata Filtering	Total
`LangChain`	3	2	3	3	3	14
`LlamaIndex`	3	3	2	2	3	13
`Haystack`	2	1	3	2	2	10

Table 2.3: Comparison of RAG Frameworks Based on Integration, Extensibility, and Ecosystem Factors

- **Integration**: LangChain and LlamaIndex offer broad support for vector DBs and embedding models, including ChromaDB, FAISS, Weaviate, OpenAI and HuggingFace [14] [15]. Haystack focuses more on FAISS and Elasticsearch [16].
- **Abstraction Quality**: LlamaIndex provides a clear, index-centric abstraction which is great for prototyping [15]. LangChain's chaining system is more modular but introduces boilerplate [14]. Haystack has a node-based pipeline that is more verbose and config-heavy [16].
- **Extensibility**: LangChain and Haystack are both highly extensible. LangChain supports custom chains and agents [14]. Haystack supports component overrides and custom modules [16]. LlamaIndex is simpler to extend but less expressive in complex use cases [15].
- **Docs & Ecosystem**: LangChain has the largest OSS community and plugin library, but documentation sometimes lags behind releases [14]. LlamaIndex has simpler docs and faster onboarding [15]. Haystack is strong in research settings but less active overall [16].
- **Metadata Filtering**: Both LangChain and LlamaIndex support metadata-based document routing [14] [15]. Haystack supports it indirectly through document store configuration [16].

Based on Table 2.3, LangChain is chosen as the RAG orchestration layer for Helikon [14]. It offers the best trade-off between flexibility, ecosystem maturity and integration coverage essential for building an adaptable assistant framework in educational workflows.

2.5 Feasibility Conclusion

The feasibility study consolidates the evaluation of all core technologies, leading to the selection of a final stack for the Helikon prototype. The chosen components are the E5 embedding model for its high

retrieval quality [2], the ChromaDB vector database for its privacy-focused and local-first architecture [9], and the LangChain framework for its mature and flexible orchestration capabilities [14]. This technology stack optimally balances state-of-the-art performance with the strict data sovereignty and low-infrastructure requirements of an educational setting. The subsequent Concept chapter will detail how these selected components are integrated into a cohesive system architecture.

Chapter 3

Concept

Overview

Helikon is a modular RAG-based assistant platform designed for the educational domain. It enables teachers to upload, manage, and query their own teaching material through an intuitive interface. The system prioritizes local-first deployment, user-level data isolation, and fast response times, while maintaining full GDPR compliance [7]. Documents move through a structured pipeline, from ingestion to vectorized retrieval, optimized for offline and privacy-focused use cases [5].

Workflow Structure

At the core of Helikon lies a modular pipeline built from reusable LangChain components [14]. Uploaded documents are automatically preprocessed, chunked, embedded using E5 [3], and stored in a local vector database (ChromaDB) [9]. Retrieval happens through semantic similarity search, followed by context-aware response generation [5]. The same pipeline can be reused for multiple use cases like Q&A, summarization, or interactive feedback generation.

API Integration

LangChain serves as the orchestration layer [14]. It connects the embedding model, vector DB, retriever, and LLM generator into a cohesive chain. All RAG operations are exposed through a backend API that acts as the system entry point. Access control is implemented through middleware that enforces permission rules (e.g., based on user roles or document tags) [20]. Depending on the deployment mode, either local LLMs or cloud APIs (OpenAI, Mistral, etc.) can be used.

Technical Infrastructure

Helikon runs on a container-based setup. All components — including ChromaDB, LangChain backend, and optional LLM interfaces — are deployed via Docker. Persistent volumes ensure embeddings and metadata remain accessible across restarts. The architecture is designed to run on school-managed servers or edge devices, removing any dependency on third-party infrastructure and supporting full data locality [7].

Security and Access Control

Although user storage and authentication are handled externally (e.g. via a PostgreSQL-based service), the system respects user context at every stage of the pipeline. Metadata like subject, grade, and ownership are embedded into documents and used during retrieval-time filtering [6]. Only users with valid rights can access matching content. This ensures data separation across roles (teacher, student, admin) and aligns with GDPR requirements [7].

A lightweight user and permission system is planned to be integrated into the backend layer. Each user (e.g. teacher, student, admin) will be associated with metadata attributes such as subject, grade, and ownership. These attributes are attached to documents during upload and used for filtering during retrieval [10]. Role-based (RBAC) and attribute-based (ABAC) models will govern access control policies [20], ensuring that only authorized users can access relevant content. While the actual identity and session management (e.g. via PostgreSQL or external auth providers) is out of scope for this work, the retrieval layer is designed to enforce these policies reliably.

Quality Assurance

All components of the system are validated through unit and integration tests. The embedding and chunking pipeline is benchmarked for consistency [3]. Retrieval accuracy and permission enforcement are tested via simulated multi-user queries. End-to-end workflow validation ensures that the assistant remains reliable, even when scaling to larger document sets [6].

User Testing and Validation

To ensure optimal usability for educators, a structured user testing phase is planned with participating teachers. Test participants will evaluate the document upload interface, query formulation, and response quality based on real classroom scenarios. Feedback will be collected through task-based observations and semi-structured interviews. This iterative validation process ensures that the system meets pedagogical requirements and remains accessible to non-technical users. Results from the usability tests will inform UI refinements and workflow optimizations before deployment.

Bibliography

[1] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China: Association for Computational Linguistics, 2019, pp. 3982–3992. doi: 10.18653/v1/D19-1410.

[2] L. Wang *et al.*, "Text Embeddings by Weakly-Supervised Contrastive Pre-training," *arXiv preprint arXiv:2212.03533*, 2022, [Online]. Available: https://arxiv.org/abs/2212.03533

[3] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual E5 Text Embeddings: A Technical Report," *arXiv preprint arXiv:2402.05672*, 2024, [Online]. Available: https://arxiv.org/abs/2402.05672

[4] H. Su *et al.*, "One Embedder, Any Task: Instruction-Finetuned Text Embeddings," in *Findings of the Association for Computational Linguistics: ACL 2023*, Toronto, Canada: Association for Computational Linguistics, 2023, pp. 1102–1121. doi: 10.18653/v1/2023.findings-acl.71.

[5] Z. Jing *et al.*, "When Large Language Models Meet Vector Databases: A Survey," *arXiv preprint arXiv:2402.01763*, 2024, [Online]. Available: https://arxiv.org/abs/2402.01763

[6] J. J. Pan, J. Wang, and G. Li, "Survey of Vector Database Management Systems," *The VLDB Journal*, vol. 33, no. 5, pp. 1591–1615, 2024, doi: 10.1007/s00778-024-00864-x.

[7] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 of the European Parliament and of the Council on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)." [Online]. Available: https://eur-lex.europa.eu/eli/reg/2016/679/oj

[9] Chroma, "ChromaDB: The AI-native Open-Source Embedding Database." [Online]. Available: https://www.trychroma.com/

[10] Weaviate B.V., "Weaviate Vector Database Documentation." [Online]. Available: https://weaviate.io/

[11] Pinecone Systems Inc., "Pinecone: The Vector Database to Build Knowledgeable AI." [Online]. Available: https://www.pinecone.io/

[12] M. Douze *et al.*, "The Faiss Library," *arXiv preprint arXiv:2401.08281*, 2024, [Online]. Available: https://arxiv.org/abs/2401.08281

[14] LangChain Inc., “LangChain: Building Applications with LLMs through Composability.” [Online]. Available: https://www.langchain.com/

[15] LlamaIndex, “LlamaIndex: Data Framework for LLM Applications.” [Online]. Available: https://www.llamaindex.ai/

[16] deepset, “Haystack: NLP Framework for Building Production-Ready LLM Applications.” [Online]. Available: https://haystack.deepset.ai/

[17] QueryPie, “RAG 2.0 Security: Microsoft and Meta's Groundwork, QueryPie Builds the Bridge.” [Online]. Available: https://www.querypie.com/features/documentation/white-paper/23/rag-security-querypie-builds-the-bridge

[18] Microsoft Azure, “Azure OpenAI Service Documentation.” [Online]. Available: https://learn.microsoft.com/en-us/azure/ai-services/openai/

[20] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-Based Access Control Models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996, doi: 10.1109/38.485845.

List of Prompts

[8] OpenAI, “PROMPT, ChatGPT GPT-5.2. write a short academic paragraph introducing vector databases in the context of embedding-based retrieval. explain how they differ from traditional databases, why they are needed, and what technical trade-offs exist. end with a sentence that transitions into a comparison of leading frameworks..” Oct. 24, 2025.

[13] OpenAI, “PROMPT, ChatGPT GPT-5.21. write a short academic paragraph introducing retrieval-augmented generation pipelines without repeating basic theory. explain their modularity, mention retriever and generator, and hint at their application in precise, interpretable systems. end with a transition into vector database technologies..” Dec. 09, 2025.

[19] OpenAI, “PROMPT, ChatGPT GPT-5.2. Erstelle mit Fletcher ein Diagramm, das zwei Ansätze zur Zugriffskontrolle in Vektordatenbanken gegenüberstellt: Links ein Shared-Index-Modell, in dem eine gemischte Vector Index (Mixed Data) über eine Metadata Filter-Stufe zu einem Result führt, rechts ein Native-Tenancy-Modell mit einem Tenant Router, der Anfragen auf getrennte Shards (z.\,B. Shard A (Class 1) und Shard B (Class 2)) verteilt, wobei nur der passende Shard durchsucht wird. Nutze einfache Formen (rect, cylinder, diamond, hexagon, parallelogram), dezente Grautöne, klare Abstände und setze die Beschriftung als Caption Comparison of Isolation Strategies: Shared Index with Metadata Filtering (Left) vs. Native Tenant Sharding (Right)..” Dec. 22, 2025.

List of Figures

List of Tables